



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Informe Final

Proyecto *Administrador de BD en consola o terminal*

Curso: Base de Datos II

Docente: Patrick Cuadros Quiroga

Integrantes:

Jahuira Pilco, Dayan Elvis ***(2022075749)***

Mamani Cori, Cristhian Carlos ***(2023077282)***

Tacna – Perú
2026

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	DEJ	CCM	PCQ	04/07/2026	Versión Original

Sistema Administrador de BD en consola o terminal

Documento de Diccionario de Datos

Versión 1.0

Nota: El sistema no cuenta con un motor de base de datos propio; actúa como intermediario hacia motores externos. Por ello, este diccionario documenta las estructuras internas de datos de la aplicación (clases conectoras, excepciones y variables de sesión).

ÍNDICE GENERAL

ÍNDICE GENERAL	3
Introducción	4
Modelo de Clases	4
Diccionario de Datos (Estructuras Internas)	4
a. Clase BaseConnector	4
b. Clase SQLiteConnector	4
c. Clase MySQLConnector	5
d. Clase PostgresConnector	5
e. Clase BaseNoSQLConnector	5
f. Clase MongoDBConnector	5
g. Clase RedisConnector	6
h. Clase CassandraConnector	6
i. Clase REPL	6
j. Clase TableFormatter	7
k. Módulo exceptions	7
l. Variables de Sesión y Configuración	7
Relaciones entre Clases	8
Reglas de Negocio	8
a. Gestión de Conexiones	8
b. Gestión de Resultados	8
c. Restricciones de las Integraciones Externas	8
d. Manejo de Errores	8
Módulos y Componentes del Sistema	9
Conclusiones	9
Recomendaciones	9
Bibliografía	9

Introducción

El presente documento describe el Diccionario de Datos del sistema “Administrador de BD en consola o terminal” (dbcli), una herramienta de línea de comandos que permite conectarse y administrar motores de bases de datos relacionales (SQLite, MySQL, PostgreSQL) y no relacionales (MongoDB, Redis, Cassandra).

A diferencia de un sistema tradicional, dbcli no incluye un motor de base de datos propio ni un esquema de almacenamiento persistente: actúa únicamente como una capa intermediaria entre el usuario y los distintos SGBD externos. Por esta razón, el presente diccionario de datos documenta las estructuras internas de la aplicación —clases conectoras, excepciones, formateador de resultados y variables de sesión— que constituyen el modelo de datos con el que trabaja el sistema durante su ejecución.

Este documento cumple la misma función que un diccionario de datos convencional: servir de referencia para desarrolladores que deseen extender el sistema (por ejemplo, agregando un nuevo conector) y para comprender cómo se organiza, transporta y descarta la información dentro de la aplicación.

Modelo de Clases

El modelo de clases se organiza en tres capas principales: la capa de presentación (REPL y TableFormatter), la capa de conectores (BaseConnector, BaseNoSQLConnector y sus implementaciones concretas) y la capa de manejo de errores (módulo exceptions). Los conectores relacionales (SQLiteConnector, MySQLConnector, PostgresConnector) y no relacionales (MongoDBConnector, RedisConnector, CassandraConnector) heredan de una clase base común, lo que permite que el REPL —y las integraciones externas de VS Code, MCP y Telegram— interactúen con cualquier motor mediante la misma interfaz, sin conocer los detalles particulares de cada SGBD. El detalle gráfico de este modelo se encuentra en el Diagrama de Clases del Documento de Arquitectura de Software (FD04, sección 3.2.5).

Diccionario de Datos (Estructuras Internas)

a. Clase BaseConnector

Clase abstracta que define el contrato común para todos los conectores relacionales. No se instancia directamente.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
connect(**kwargs)	Establece la conexión con el motor de base de datos usando los parámetros recibidos (db, user, password, host, etc.)	bool	N/A	Método abstracto, obligatorio en subclases
disconnect()	Cierra la conexión activa y libera los recursos asociados	bool	N/A	Método abstracto
execute_query(query)	Ejecuta una instrucción SQL sobre la conexión activa	Tuple[headers, rows, rowcount]	N/A	Método abstracto
get_tables()	Retorna el listado de tablas existentes en la base de datos conectada	List[str]	N/A	Método abstracto
get_type()	Retorna el nombre del motor de base de datos (SQLite, MySQL, PostgreSQL)	str	N/A	Método abstracto
get_info()	Retorna información resumida de la conexión activa (host, usuario, BD)	str	N/A	Método abstracto

b. Clase SQLiteConnector

Hereda de BaseConnector. Gestiona la conexión a archivos de base de datos SQLite locales.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
ruta	Ruta del archivo de base de datos SQLite (.db)	str	N/A	Obligatorio en connect()
connect(ruta)	Abre o crea el archivo SQLite indicado	bool	N/A	Override de BaseConnector

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
execute_query(query)	Ejecuta sentencias SQL (SELECT, INSERT, UPDATE, DELETE, CREATE, DROP) sobre el archivo	Tuple	N/A	Override
get_tables()	Lista las tablas contenidas en el archivo SQLite	List[str]	N/A	Override

c. Clase MySQLConnector

Hereda de BaseConnector. Gestiona la conexión a servidores MySQL mediante la librería mysql-connector-python.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
db	Nombre de la base de datos MySQL a la que se desea conectar	str	N/A	Obligatorio
user	Usuario de conexión al servidor	str	N/A	Obligatorio
password	Contraseña del usuario, ingresada en texto plano	str	N/A	Obligatorio
host	Dirección del servidor MySQL	str	N/A	Opcional, por defecto localhost
connect(db, user, password, host)	Establece la conexión con el servidor MySQL indicado	bool	N/A	Override de BaseConnector
execute_query(query)	Ejecuta sentencias SQL sobre la conexión MySQL activa	Tuple	N/A	Override

d. Clase PostgresConnector

Hereda de BaseConnector. Gestiona la conexión a servidores PostgreSQL mediante la librería psycopg2.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
db	Nombre de la base de datos PostgreSQL a la que se desea conectar	str	N/A	Obligatorio
user	Usuario de conexión al servidor	str	N/A	Obligatorio
password	Contraseña del usuario, ingresada en texto plano	str	N/A	Obligatorio
host	Dirección del servidor PostgreSQL	str	N/A	Opcional, por defecto localhost
connect(db, user, password, host)	Establece la conexión con el servidor PostgreSQL indicado	bool	N/A	Override de BaseConnector
execute_query(query)	Ejecuta sentencias SQL sobre la conexión PostgreSQL activa	Tuple	N/A	Override

e. Clase BaseNoSQLConnector

Clase abstracta que hereda de BaseConnector y extiende el contrato para motores no relacionales.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
list_collections()	Retorna las colecciones, keyspaces o bases lógicas del motor NoSQL conectado	List[str]	N/A	Método abstracto

f. Clase MongoDBConnector

Hereda de BaseNoSQLConnector. Gestiona la conexión a MongoDB mediante la librería pymongo.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
db	Nombre de la base de datos Mongo a la que se desea conectar	str	N/A	Obligatorio
host	Dirección del servidor MongoDB	str	N/A	Opcional, por defecto localhost

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
puerto	Puerto de conexión al servidor MongoDB	int	5	Opcional, por defecto 27017
find(coleccion, json_filtro)	Consulta documentos de una colección que cumplen el filtro JSON indicado	List[dict]	N/A	Equivalente a SELECT; solo lectura en MCP/Telegram
insert(coleccion, json_doc)	Inserta un nuevo documento en la colección indicada	bool	N/A	No disponible en MCP/Telegram
update(coleccion, filtro, set)	Actualiza los documentos que cumplen el filtro indicado	int (docs. afectados)	N/A	No disponible en MCP/Telegram
delete(coleccion, json_filtro)	Elimina los documentos que cumplen el filtro indicado	int (docs. afectados)	N/A	No disponible en MCP/Telegram

g. Clase RedisConnector

Hereda de BaseNoSQLConnector. Gestiona la conexión a Redis mediante la librería redis-py.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
db_index	Índice de base de datos lógica de Redis (0 a 15)	int	2	Opcional, por defecto 0
host	Dirección del servidor Redis	str	N/A	Opcional, por defecto localhost
puerto	Puerto de conexión al servidor Redis	int	4	Opcional, por defecto 6379
set(clave, valor)	Asigna un valor de tipo cadena a una clave	bool	N/A	No disponible en MCP/Telegram
get(clave)	Obtiene el valor asociado a una clave	str	N/A	Equivalente a SELECT; solo lectura en MCP/Telegram
del(clave)	Elimina una clave del almacén Redis	bool	N/A	No disponible en MCP/Telegram
keys(patron)	Lista las claves que cumplen un patrón de búsqueda	List[str]	N/A	Solo lectura en MCP/Telegram

h. Clase CassandraConnector

Hereda de BaseNoSQLConnector. Gestiona la conexión a Cassandra mediante la librería cassandra-driver.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
keyspace	Espacio de claves (keyspace) de Cassandra al que se desea conectar	str	N/A	Obligatorio
host	Nodo del clúster Cassandra	str	N/A	Opcional, por defecto localhost
execute_query(cql)	Ejecuta sentencias CQL (SELECT, INSERT, UPDATE, DELETE)	Tuple	N/A	Override de BaseConnector
get_tables()	Lista las tablas definidas dentro del keyspace conectado	List[str]	N/A	Override

i. Clase REPL

Clase principal del sistema. Implementa el bucle de lectura, interpretación y ejecución de comandos (Read-Eval-Print Loop).

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
running	Indica si el bucle principal del REPL sigue activo	bool	1	NOT NULL, valor inicial True
modo	Modo de operación seleccionado al inicio (Relacional o NoSQL)	str	N/A	NOT NULL, definido en CU001
conector	Instancia activa del conector actualmente conectado	BaseConnector	N/A	Nulo si no existe conexión activa; una sola instancia a la vez
formateador	Instancia de TableFormatter usada para mostrar resultados en consola	TableFormatter	N/A	NOT NULL

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
last_results	Resultado (filas y cabeceras) de la última consulta SELECT/find ejecutada	List[Tuple]	N/A	Volátil; se descarta al ejecutar una nueva consulta o cerrar el programa
run()	Inicia el bucle principal de lectura de comandos del usuario	void	N/A	
execute(comando)	Analiza el comando ingresado y lo despacha al manejador correspondiente	void	N/A	
_connect(parametros)	Maneja el comando 'connect' e instancia el conector adecuado	void	N/A	
_select(parametros)	Maneja los comandos 'select'/'find' y almacena el resultado en last_results	void	N/A	
_insert(parametros)	Maneja los comandos 'insert into'/'insert'	void	N/A	
_update(parametros)	Maneja los comandos 'update'	void	N/A	
_delete(parametros)	Maneja los comandos 'delete'/'del'	void	N/A	
_export(archivo)	Exporta el contenido de last_results a un archivo CSV	void	N/A	Requiere una consulta SELECT previa
_help()	Muestra los comandos disponibles según el modo activo (relacional o NoSQL)	void	N/A	
_status()	Muestra el estado de la conexión activa (tipo de motor, host, base de datos)	void	N/A	

j. Clase TableFormatter

Se encarga de dar formato tabular a los resultados de las consultas, utilizando la librería Rich.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
print_table(datos, filas)	Formatea y muestra en consola los resultados recibidos, incluyendo el total de filas	void	N/A	NOT NULL

k. Módulo exceptions

Contiene las excepciones personalizadas utilizadas para el manejo estructurado de errores en toda la aplicación.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
ConnectionError	Excepción lanzada ante fallos al establecer o mantener la conexión con el motor de base de datos	Exception	N/A	Hereda de Exception
SyntaxError	Excepción lanzada cuando un comando ingresado no respeta la sintaxis básica esperada	Exception	N/A	Hereda de Exception
QueryError	Excepción lanzada cuando la ejecución de una consulta falla en el motor de base de datos	Exception	N/A	Hereda de Exception

l. Variables de Sesión y Configuración

Datos que existen únicamente en memoria durante la ejecución del programa y que no se persisten entre sesiones.

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
last_results	Resultado de la última consulta SELECT/find ejecutada, disponible para exportación a CSV	List[Tuple]	N/A	Volátil; se descarta al cerrar el programa o ejecutar una nueva consulta

Campo	Descripción	Tipo de Dato	Longitud	Restricciones
credenciales_conexion	Parámetros de conexión ingresados por el usuario en el comando connect (db, user, password, host)	dict	N/A	No se almacena en disco ni en logs; se solicita en cada inicio de sesión
modo_operacion	Modo seleccionado al iniciar el programa (Relacional / NoSQL)	str	N/A	Determina los comandos disponibles en help()

Relaciones entre Clases

- El REPL mantiene una única instancia activa de un conector (BaseConnector) a la vez; para cambiar de motor, primero debe desconectarse.
- SQLiteConnector, MySQLConnector y PostgresConnector heredan de BaseConnector e implementan sus métodos abstractos según el motor relacional correspondiente.
- MongoDBConnector, RedisConnector y CassandraConnector heredan de BaseNoSQLConnector, que a su vez extiende BaseConnector.
- El REPL utiliza una instancia de TableFormatter para dar formato a los resultados devueltos por cualquier conector antes de mostrarlos en consola.
- Los errores generados por los conectores (ConnectionError, SyntaxError, QueryError) son capturados por el REPL, que muestra el mensaje correspondiente sin detener la ejecución.
- La variable last_results se llena únicamente tras una consulta de lectura (select/find/get) y es consumida por el comando export.
- El servidor MCP y el bot de Telegram reutilizan la misma capa de conectores que el REPL, pero restringen su uso a los métodos de solo lectura (execute_query de tipo SELECT, find, get, get_tables, list_collections).

Reglas de Negocio

a. Gestión de Conexiones

- El sistema no incluye ningún motor de base de datos propio; el usuario debe tener instalado y accesible al menos un motor compatible.
- Solo puede existir una instancia de conector activa a la vez; para conectarse a un nuevo motor, el usuario debe desconectarse primero (regla reflejada en el atributo 'conector' de la clase REPL).
- Las credenciales de conexión no se almacenan ni se recuerdan entre sesiones; deben ingresarse manualmente en cada ejecución mediante el comando connect.

b. Gestión de Resultados

- La variable last_results solo se actualiza tras ejecutar una consulta de lectura (select/find/get) y es volátil: se descarta al cerrar el programa o al ejecutar una nueva consulta.
- El comando export solo tiene efecto si existe un valor previo en last_results; de lo contrario, no genera ningún archivo.

c. Restricciones de las Integraciones Externas

- Las herramientas expuestas al servidor MCP y al bot de Telegram están restringidas a operaciones de solo lectura (execute_query SELECT, find, get, get_tables, list_collections).
- Cualquier intento de ejecutar un comando de escritura desde el servidor MCP o el bot de Telegram es rechazado automáticamente, independientemente del cliente que lo origine.

d. Manejo de Errores

- Todo error de sintaxis, conexión o ejecución de consulta es capturado mediante las excepciones `ConnectionError`, `SyntaxError` y `QueryError`, y mostrado al usuario sin finalizar el programa.

Módulos y Componentes del Sistema

Dado que el sistema no posee objetos de base de datos propios (triggers, procedimientos almacenados o eventos), esta sección documenta los módulos internos de la aplicación que cumplen un rol equivalente en la organización de la lógica y los datos.

Nombre	Tipo	Módulo Relacionado	Descripción
main.py	Módulo	Aplicación dbcli	Punto de entrada de la aplicación; inicializa el REPL
repl.py	Módulo	REPL	Contiene la clase REPL y el bucle principal de comandos
help.py	Módulo	REPL	Genera el panel de ayuda según el modo de operación activo
export.py	Módulo	REPL	Exporta el contenido de <code>last_results</code> a un archivo CSV
table_formatter.py	Módulo	TableFormatter	Formatea los resultados de las consultas para su presentación en consola
exceptions.py	Módulo	Manejo de Errores	Define las excepciones personalizadas <code>ConnectionError</code> , <code>SyntaxError</code> y <code>QueryError</code>
connectors/base.py	Módulo	BaseConnector / BaseNoSQLConnector	Define las clases base abstractas para todos los conectores
connectors/relacionales/*	Módulo	SQLiteConnector, MySQLConnector, PostgresConnector	Implementaciones concretas de conectores relacionales
connectors/nosql/*	Módulo	MongoDBConnector, RedisConnector, CassandraConnector	Implementaciones concretas de conectores NoSQL

Conclusiones

- El diccionario de datos permitió documentar la estructura interna de una aplicación que, al no poseer una base de datos propia, organiza su información en clases conectoras, variables de sesión volátiles y un módulo de excepciones.
- La herencia común entre los conectores (`BaseConnector` y `BaseNoSQLConnector`) garantiza que el REPL y las integraciones externas (VS Code, MCP, Telegram) trabajen con una interfaz uniforme, independientemente del motor de base de datos conectado.
- La variable `last_results` concentra el único dato persistente durante la sesión, evidenciando el carácter de intermediario —y no de almacén— que tiene el sistema.
- Las restricciones de solo lectura aplicadas al servidor MCP y al bot de Telegram quedan reflejadas directamente en las restricciones documentadas para los métodos de cada conector.

Recomendaciones

- Mantener actualizado este diccionario cada vez que se agregue un nuevo conector, heredando obligatoriamente de `BaseConnector` o `BaseNoSQLConnector`.
- Documentar explícitamente en el código los tipos de retorno de los métodos abstractos, para facilitar la extensión del sistema por nuevos desarrolladores.
- Evaluar la incorporación de un esquema de configuración (archivo `.env` o similar) para credenciales, sin comprometer la regla de negocio de no almacenarlas en disco en texto plano.

- Para versiones futuras, considerar el registro (logging) de las operaciones de solo lectura realizadas desde el servidor MCP y el bot de Telegram, sin almacenar credenciales.

Bibliografía

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7ª ed.). McGraw-Hill Education.

Python Software Foundation. (2026). The Python Standard Library. Recuperado de <https://docs.python.org/3/>

Pallets/Rich Project. (2026). Rich Documentation. Recuperado de <https://rich.readthedocs.io/>